

Grafana & Observability

Java Backend Interview — Short Notes

1. Observability — The Three Pillars

- **Observability:** ability to understand internal state of a system from its external outputs. Beyond monitoring.
- Monitoring tells you *when* something is wrong. Observability helps you understand *why*.

Pillar	What It Is	Tools	Realistic Use Case
Metrics	Numeric measurements over time. Aggregated. Low storage cost.	Prometheus, Micrometer, StatsD, CloudWatch, Datadog	JVM heap usage, HTTP request rate, DB connection pool size, error rate, p99 latency.
Logs	Timestamped records of discrete events. Text-based. High detail.	ELK Stack (Elasticsearch + Logstash + Kibana), Loki + Grafana, Splunk, CloudWatch Logs	Exception stack traces, request/response payloads, audit events, security access logs.
Traces	End-to-end request journey across services. Timing per span.	Jaeger, Zipkin, Tempo, AWS X-Ray, Datadog APM, OpenTelemetry	Trace HTTP request through API → Order Service → Payment Service → DB. Find which service is slow.

✓ Use all three together. Metric alert fires → logs show what happened → trace shows where in the system. Correlation via trace ID.

2. Prometheus — Metrics Collection

2.1 How Prometheus Works

- **Pull model:** Prometheus scrapes /actuator/prometheus endpoint on each service at configured interval.
- **Time-series DB:** stores metric samples as (timestamp, value) with labels (key=value pairs).
- **Labels:** dimensions for filtering/grouping. service='order', env='prod', method='POST', status='200'.
- **Retention:** default 15 days local. Long-term storage: Thanos, Cortex, VictoriaMetrics, Mimir.

2.2 Metric Types

Type	Description	Use Case	Example
Counter	Monotonically increasing value. Never decreases. Resets on restart.	Total requests, total errors, bytes sent.	http_requests_total{method='POST', status='200'}
Gauge	Can go up and down. Current value snapshot.	Current connections, memory usage, queue depth, temperature.	jvm_memory_used_bytes{area='heap'}
Histogram	Observe values in configurable buckets. Calculates quantiles.	Request duration, response size. Accurate quantiles.	http_request_duration_seconds_bucket{le='0.5'}
Summary	Like histogram but pre-calculated quantiles on client side.	Accurate per-instance quantiles. Less aggregatable across instances.	rpc_duration_seconds{quantile='0.99'}

2.3 PromQL — Prometheus Query Language

Query	What It Returns
-------	-----------------

http_requests_total	Current counter value for all label combinations.
rate(http_requests_total[5m])	Per-second average request rate over last 5 minutes. Use rate() on counters.
sum(rate(http_requests_total[5m])) by (service)	Total RPS grouped by service.
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))	p99 latency over last 5 minutes.
jvm_memory_used_bytes / jvm_memory_max_bytes * 100	JVM heap usage percentage.
increase(http_requests_total{status=~'5..'}[1h])	Total 5xx errors in last hour.
topk(5, rate(http_requests_total[5m]))	Top 5 highest-traffic endpoints.
absent(up{job='order-service'})	Alert when order-service target disappears from scrape.

2.4 Spring Boot + Micrometer + Prometheus

```
# pom.xml dependencies
spring-boot-starter-actuator
micrometer-registry-prometheus

# application.yml
management:
  endpoints:
  web:
  exposure:
include: health,info,prometheus,metrics
metrics:
tags:
application: order-service
environment: production

// Custom metrics in Java
@Component
public class OrderMetrics {
  private final Counter ordersCreated;
  private final Gauge pendingOrders;
  private final Timer orderProcessingTime;

  public OrderMetrics(MeterRegistry registry, OrderRepository repo) {
    ordersCreated = Counter.builder("orders.created")
      .tag("currency", "USD")
      .description("Total orders created")
      .register(registry);
    pendingOrders = Gauge.builder("orders.pending", repo,
      r -> r.countByStatus("PENDING"))
      .register(registry);
    orderProcessingTime = Timer.builder("orders.processing.time")
      .publishPercentiles(0.5, 0.95, 0.99)
      .register(registry);
  }

  public void recordOrderCreated() { ordersCreated.increment(); }
  public void recordProcessingTime(Runnable task) {
    orderProcessingTime.record(task);
  }
}
```

3. Grafana — Visualisation & Dashboards

3.1 Grafana Core Concepts

Concept	Description	Notes
Data Source	Connection to metrics/logs/traces backend.	Prometheus, Loki, Tempo, Elasticsearch, CloudWatch, PostgreSQL, MySQL, InfluxDB, Jaeger.
Dashboard	Collection of panels. JSON-based — version controlled in Git.	Import community dashboards from grafana.com/dashboards (Spring Boot #12900).
Panel	Individual visualisation: graph, gauge, stat, table, heatmap, logs, traces.	Each panel has its own query, legend, thresholds, overrides.
Variable	Dynamic dashboard filter (service, environment, pod).	<code>\$service</code> variable → filter all panels at once.
Alert	Alert rule on metric query. Routes to contact points (Slack, PagerDuty, email).	Grafana Alerting or Prometheus Alertmanager. Alert state: Normal → Pending → Firing.
Annotation	Mark events on graph (deployment, incident, restart).	Correlate metric spikes with deployments. Automated via CI/CD webhook.
Playlist	Rotate dashboards automatically.	NOC / ops screens — cycle through services.
Provisioning	Define dashboards and data sources as YAML/JSON files.	GitOps for Grafana — dashboards in code, no manual UI config.

3.2 Essential Panels for Java Microservice Dashboard

Panel	Metric Query	What to Watch
Request Rate (RPS)	<code>sum(rate(http_server_requests_seconds_count[5m])) by (uri)</code>	Sudden drops = outage. Spikes = load or attack.
Error Rate	<code>sum(rate(http_server_requests_seconds_count{status=~'5..'}[5m]))</code>	Should be near 0. Any spike warrants investigation.
p99 Latency	<code>histogram_quantile(0.99, rate(http_server_requests_seconds_bucket[5m]))</code>	SLA target: e.g., <500ms. Alert if exceeded.
JVM Heap Usage	<code>jvm_memory_used_bytes{area='heap'} / jvm_memory_max_bytes{area='heap'}</code>	Alert >80%. Increasing trend = memory leak.
GC Pause Time	<code>rate(jvm_gc_pause_seconds_sum[5m])</code>	High GC = memory pressure. Check heap size and GC tuning.
Thread Pool	<code>executor_active_threads / executor_pool_max_threads</code>	Near 100% = thread starvation. Scale or tune pool.
DB Connection Pool	<code>hikaricp_connections_active / hikaricp_connections_max</code>	Near max = DB bottleneck. Check slow queries.
Kafka Consumer Lag	<code>kafka_consumer_group_lag</code>	Growing lag = consumer falling behind. Scale consumers.

4. Alerting with Prometheus Alertmanager

4.1 Alert Rules (Prometheus)

```
# prometheus-rules.yaml
groups:
- name: order-service
rules:
- alert: HighErrorRate
expr: |
sum(rate(http_server_requests_seconds_count{status=~'5..'}[5m]))
/
sum(rate(http_server_requests_seconds_count[5m])) > 0.05
for: 2m # must be true for 2 min before firing
labels:
severity: critical
team: backend
annotations:
summary: 'High 5xx error rate on order-service'
description: 'Error rate {{ $value | humanizePercentage }}'
- alert: HighP99Latency
expr: |
histogram_quantile(0.99,
rate(http_server_requests_seconds_bucket[5m])) > 1.0
for: 5m
labels: { severity: warning }
annotations:
summary: 'p99 latency over 1 second'
- alert: JVMHeapCritical
expr: |
jvm_memory_used_bytes{area='heap'}
/ jvm_memory_max_bytes{area='heap'} > 0.90
for: 3m
labels: { severity: critical }
```

4.2 Alertmanager Routing

```
# alertmanager.yaml
route:
group_by: [alertname, service]
group_wait: 30s # wait before first notification
group_interval: 5m # how often to re-notify
repeat_interval: 4h # resend if still firing
receiver: slack-critical
routes:
- match: { severity: critical }
receiver: pagerduty-oncall
- match: { team: backend }
receiver: slack-backend
receivers:
- name: pagerduty-oncall
pagerduty_configs:
- routing_key: <PD_KEY>
- name: slack-backend
slack_configs:
- api_url: <WEBHOOK>
channel: '#backend-alerts'
```

```
text: '{{ .CommonAnnotations.summary }}'
```

5. Grafana Observability Stack

Tool	Role	Key Feature	Query Language
Prometheus	Metrics collection and storage.	Pull-based scraping, powerful alerting.	PromQL
Grafana	Visualisation, dashboards, alerting UI.	Multi-source, team collaboration, provisioning.	Uses data source QL
Loki	Log aggregation. Like Prometheus for logs.	Label-based indexing. Low cost (index labels not full text).	LogQL
Tempo	Distributed tracing backend.	Stores traces. Query by trace ID or span attributes.	TraceQL
OpenTelemetry	Vendor-neutral instrumentation standard.	One SDK for metrics + logs + traces. Replaces Micrometer + Sleuth.	OTLP protocol
Mimir	Long-term metrics storage (Prometheus scale-out).	Horizontally scalable Prometheus-compatible backend.	PromQL
Alertmanager	Alert routing, deduplication, grouping.	Routes alerts to Slack/PagerDuty/email. Silences and inhibition.	YAML config

Loki LogQL examples

```
{app='order-service', env='prod'} |= 'ERROR'
```

```
{app='order-service'} | json | level='error' | line_format '{{.message}}'
```

Count error lines per minute

```
sum(rate({app='order-service'} |= 'ERROR' [1m])) by (pod)
```

5.1 OpenTelemetry with Spring Boot

```
# Add to pom.xml – auto-instrument HTTP, DB, messaging
opentelemetry-spring-boot-starter

# application.yml
otel:
  service-name: order-service
  exporter:
  otlp:
    endpoint: http://otel-collector:4318
  traces:
    sampler: parentbased_traceidratio
    sampler-arg: 0.1 # sample 10% of traces in prod
// Add custom span attributes
Span span = tracer.spanBuilder("processOrder").startSpan();
try (Scope scope = span.makeCurrent()) {
  span.setAttribute("order.id", orderId);
  span.setAttribute("order.total", total.toString());
  processOrder(orderId);
} catch (Exception e) {
  span.recordException(e);
  span.setStatus(StatusCode.ERROR, e.getMessage());
  throw e;
} finally { span.end(); }
```

6. Interview Questions

Observability

Q1. What is the difference between monitoring and observability?

→ Monitoring: watch predefined metrics, alert when threshold crossed. Tells you something is wrong. Observability: ability to understand WHY from system outputs. Involves metrics, logs, and traces together. With monitoring you know the service is down. With observability you understand which component failed, at what step, with what error, and how long it took.

Q2. Explain the three pillars of observability and how they complement each other.

→ Metrics: numeric aggregates — fast to query, cheap to store. Tell you WHAT is wrong (high error rate, slow latency). Logs: detailed event records — tell you exactly WHAT happened at a specific time. Traces: show request flow across services — tell you WHERE in the distributed system the problem is. Together: metric alert → filter logs by time range → correlate with trace ID to find root cause.

Q3. What is OpenTelemetry?

→ Vendor-neutral CNCF standard for observability instrumentation. Provides unified SDK for collecting metrics, logs, and traces. Single instrumentation exports to any backend (Jaeger, Prometheus, Datadog, Grafana Tempo). Replaces proprietary agents and multiple libraries. Spring Boot 3.x has native OpenTelemetry support via Micrometer Tracing.

Prometheus & Grafana

Q4. Explain PromQL rate() vs increase().

→ rate(): per-second average rate of change over time range. Use for gauging speed. Example: rate(http_requests_total[5m]) = requests per second averaged over 5 minutes. increase(): total increase in counter over time range. Example: increase(http_requests_total[1h]) = total requests in last hour. Both account for counter resets.

Q5. What is the difference between Histogram and Summary in Prometheus?

→ Histogram: configurable buckets defined server-side. Supports aggregation across instances (sum histograms, then calculate quantile). Accurate for aggregated infrastructure-wide percentiles. Summary: calculates quantiles on client side. Accurate per-instance quantiles but CANNOT be aggregated across replicas. Use Histogram for distributed systems — you can calculate p99 across all pods.

Q6. How do you set up alerting for a Java microservice in production?

→ (1) Instrument with Micrometer → expose /actuator/prometheus. (2) Prometheus scrapes metrics. (3) Write alert rules: error rate >5%, p99 latency >1s, heap >85%, consumer lag growing. (4) Alertmanager routes: warning → Slack, critical → PagerDuty. (5) for: 2m prevents flapping alerts. (6) Grafana dashboard shows metric trends correlated with alert firing times.

Q7. What is Loki and how does it differ from Elasticsearch?

→ Loki: log aggregation inspired by Prometheus. Indexes only labels (pod, service, env) not full log content. Cheap storage, fast label queries. Query: LogQL (similar to PromQL). Elasticsearch (ELK): full inverted index of every word in every log. Full-text search anywhere in log. More powerful for complex search but expensive at scale (index = 10-15% of log volume). Loki for operational logs, ELK for security/compliance/full-text search.